



BURSA TEKNİK ÜNİVERSİTESİ

BURSA TEKNİK ÜNİVERSİTESİ BİLGİSAYAR
MÜHENDİSLİĞİ BÖLÜMÜ

BLM0121 NESNEYE YÖNELİK PROGRAMLAMA DERSİ
PROJE ÖDEVİ KONU: BANKA OTOMASYON SİSTEMİ
SİMÜLASYONU

AD – SOYAD: Emin Eren KADIOĞLU

Öğrenci NO: 25360859011

Dersin Öğretmeni: Doç. Dr. Ergün GÜMÜŞ

PROJENİN AMACI VE MİMARİ YAPI

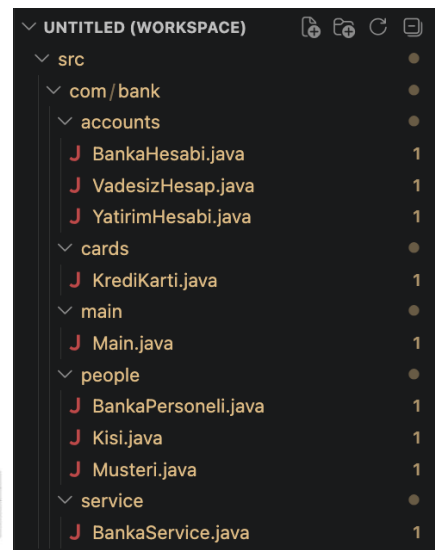
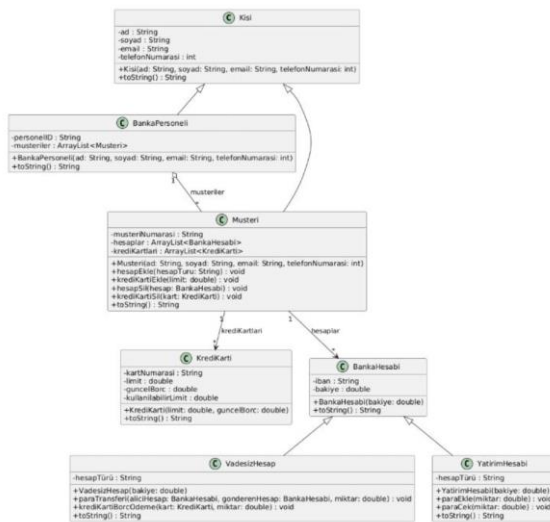
PROJENİN AMACI VE KAPSAMI:

Bu proje geliştirilirken temel amaç, derste öğrenilen nesne yönelimli programlama prensiplerini gerçek bir hayat senaryosu üzerinde denemektir. Bir bankadaki personel, müşteri, vadesiz ve yatırım hesapları arasındaki ilişkiler modellenerek; bakiye transferi, borç ödeme ve güvenlik kontrollerini içeren modüler bir yapı kurulmaya çalışılmıştır.

PROJE MİMARİSİ VE PAKET YAPISI:

Proje istenilen formatta 5 ana paket altında toplandı:

- **com.bank.app.people:** Kisi, Musteri ve BankaPersoneli sınıflarını içerir.
- **com.bank.app.accounts:** BankaHesabi, VadesizHesap ve YatirimHesabi sınıflarını barındırır.
- **com.bank.app.cards:** Kredi kartı işlemlerini yöneten KrediKarti sınıfını tutar.
- **com.bank.app.service:** İş mantığını ve atamaları koordine eden BankaService sınıfını içerir.
- **com.bank.app.main:** Uygulamanın giriş noktası olan ve tüm senaryoyu test eden Main sınıfını barındırır.



Şekil 1.0: Proje Temel UML Diyagramı ve Paket Hiyerarşisi

SINIFLARIN ANALİZİ VE KOD DETAYLARI

1. com.bank.app.people Paketi:

Bu paket, sistemdeki insan faktörünü temsil eden sınıfları barındırır. Projenin kalıtım (inheritance) yapısının temelini oluşturur; böylece personeller ve müşteriler ortak özelliklerini tek bir merkezden alarak kod tekrarını engeller.

1.1 Kisi.java Sınıfı:

Sistemin temel direği olan bu sınıf; bankadaki personellerin ve müşterilerin sahip olduğu ad, soyad, email ve telefon numarası gibi ortak özellikleri tek bir yerde toplamak amacıyla tasarlanmıştır. Nesneye Yönelik Programlama'nın kapsülleme (encapsulation) ilkesine sadık kalınarak tüm değişkenler private olarak tanımlanmış ve bu verilere erişim yalnızca Getter/Setter metotlarıyla sağlanmıştır.

```
src > com > bank > people > J Kisi.java > Kisi
1 package com.bank.app.people;
2
3 public class Kisi {
4     // OOP'nin Encapsulation (Kapsülleme) kuralı gereği değişkenleri private tanımladım.
5     // Böylece bu verilere dışarıdan doğrudan erişilmesini engelledim.
6     private String ad;
7     private String soyad;
8     private String email;
9     private int telefonNumarasi;
10
11     // Sınıftan nesne üretildiğinde ilk çalışacak olan Constructor (Yapıcı) metot.
12     // Dışarıdan alınan bilgileri sınıfın içindeki değişkenlere atıyor.
13     public Kisi(String ad, String soyad, String email, int telefonNumarasi) {
14         this.ad = ad;
15         this.soyad = soyad;
16         this.email = email;
17         this.telefonNumarasi = telefonNumarasi;
18     }
19
20     // Private değişkenlere güvenli bir şekilde erişmek ve değiştirmek için Getter ve Setter metotlarını yazdım.
21     public String getAd() { return ad; }
22     public void setAd(String ad) { this.ad = ad; }
23
24     public String getSoyad() { return soyad; }
25     public void setSoyad(String soyad) { this.soyad = soyad; }
26
27     public String getEmail() { return email; }
28     public void setEmail(String email) { this.email = email; }
29
30     public int getTelefonNumarasi() { return telefonNumarasi; }
31     public void setTelefonNumarasi(int telefonNumarasi) { this.telefonNumarasi = telefonNumarasi; }
32
33     // Ekrana yazdırırken bellek adresi yerine anlamlı bir metin dönmesi için toString metodunu ezdim (override ettim).
34     @Override
35     public String toString() {
36         return "Ad: " + ad + ", Soyad: " + soyad + ", Email: " + email + ", Tel: " + telefonNumarasi;
37     }
38 }
```

Şekil 1.1: Kisi Sınıfı ve Temel Kapsülleme Yapısı

1.2 BankaPersoneli.java Sınıfı

Banka çalışanlarını modelleyen bu sınıf, Kisi sınıfından miras (extends) almaktadır. İçerisinde yer alan ArrayList yapısı sayesinde, personelin sorumlu olduğu müşterileri tek tek takip edebilmesi sağlanmıştır. Ayrıca personellere PER-XXXX formatında sistem tarafından otomatik ve benzersiz bir ID atanmasını sağlayan rastgele bir ID üretici metot kurgulanmıştır.

```

src > com > bank > people > J BankaPersoneli.java > BankaPersoneli
1  package com.bank.app.people;
2
3  import java.util.ArrayList;
4  import java.util.Random;
5
6  // BankaPersoneli, Kisi sınıfının ortak özelliklerini Inheritance (Kalıtım) ile miras alır.
7  public class BankaPersoneli extends Kisi {
8      private String personelID;
9
10     // Bir personelin birden fazla müşterisi olabileceği için bunları bir ArrayList içinde tutuyorum.
11     private ArrayList<Musteri> musteriler;
12
13     public BankaPersoneli(String ad, String soyad, String email, int telefonNumarasi) {
14         // super() anahtar kelimesi ile üst sınıfın (Kisi) constructor'ına bilgileri gönderiyorum.
15         super(ad, soyad, email, telefonNumarasi);
16
17         // Proje isterlerine uygun olarak ID'yi dışarıdan almak yerine içeride otomatik üretiriyorum.
18         this.personelID = rastgeleIDUret();
19
20         // NullPointerException hatası almamak için listeyi başlatıyorum.
21         this.musteriler = new ArrayList<>();
22     }
23
24     // Personel ID'sini rastgele türeten yardımcı (private) metod.
25     private String rastgeleIDUret() {
26         Random rnd = new Random();
27         // PER-1234 gibi bir format oluşturuyor.
28         return "PER-" + (1000 + rnd.nextInt(bound: 9000));
29     }
30
31     // İlgili personele yeni bir müşteri atamak için kullandığım metod.
32     public void musteriekle(Musteri muster) {
33         this.musteriler.add(muster);
34     }
35
36     public String getPersonelID() { return personelID; }
37     public ArrayList<Musteri> getMusteriler() { return musteriler; }
38
39     @Override
40     public String toString() {
41         // Üst sınıfın toString metoduna Personel ID'yi de ekleyerek döndürüyorum.
42         return super.toString() + ", Personel ID: " + personelID;
43     }
44 }

```

Şekil 1.2: BankaHesabi Üst Sınıfı ve Otomatik IBAN Üretim Mantığı

1.3 Musteri.java Sınıfı

Yine Kisi sınıfından miras alan müşteri sınıfı, bankacılık işlemlerinin ve veri listelerinin merkezidir. Müşterinin sahip olduğu vadesiz/yatırım hesapları ve kredi kartları bu sınıfın içindeki ArrayList'lerde saklanır. Sistemin en kritik güvenlik önlemlerinden olan "bakiyesi bulunan bir hesabın" veya "borcu ödenmemiş bir kredi kartının" silinmesini engelleyen kontrol (if-else) algoritmaları bu sınıf içerisinde yapılandırılmıştır.

```

src > com > bank > people > J Musteri.java > Musteri
1 package com.bank.app.people;
2
3 import com.bank.app.accounts.BankaHesabi;
4 import com.bank.app.accounts.VadesizHesap;
5 import com.bank.app.accounts.YatirimHesabi;
6 import com.bank.app.cards.KrediKarti;
7
8 import java.util.ArrayList;
9 import java.util.Random;
10
11 // Musteri sınıfı da Kisi sınıfından miras alıyor.
12 public class Musteri extends Kisi {
13     private String musterNumarasi;
14
15     // Bir müşterinin birden fazla banka hesabı ve kredi kartı olabileceği için ArrayList kullandım.
16     private ArrayList<BankaHesabi> hesaplar;
17     private ArrayList<KrediKarti> krediKartlari;
18
19     public Musteri(String ad, String soyad, String email, int telefonNumarasi) {
20         super(ad, soyad, email, telefonNumarasi);
21
22         // Müşteri numarasını projede istendiği gibi rastgele üretiyorum.
23         this.musterNumarasi = rastgeleMusteriNoUret();
24         this.hesaplar = new ArrayList<>();
25         this.krediKartlari = new ArrayList<>();
26     }
27
28     // 5 haneli rastgele bir müşteri numarası üretiyor (Örn: MUS-45892)
29     private String rastgeleMusteriNoUret() {
30         Random rnd = new Random();
31         return "MUS-" + (10000 + rnd.nextInt(bound: 90000));
32     }
33
34     // Gelen parametreye göre müşteriye Vadesiz veya Yatırım hesabı açan metod.
35     public void hesapEkle(String hesapTuru) {
36         if (hesapTuru.equalsIgnoreCase(anotherString: "Vadesiz")) {
37             // Başlangıç bakiyesini 0.0 vererek yeni bir VadesizHesap nesnesi oluşturup listeye ekliyorum.
38             hesaplar.add(new VadesizHesap(bakiye: 0.0));
39             System.out.println(x: "Vadesiz hesap başarıyla eklendi.");
40         } else if (hesapTuru.equalsIgnoreCase(anotherString: "Yatirim")) {
41             hesaplar.add(new YatirimHesabi(bakiye: 0.0));
42             System.out.println(x: "Yatırım hesabı başarıyla eklendi.");
43         } else {
44             System.out.println(x: "Geçersiz hesap türü!");
45         }
46     }
47
48     // Müşteriye yeni bir kredi kartı nesnesi oluşturup ekleyen metod.
49     public void krediKartiEkle(double limit) {
50         KrediKarti yeniKart = new KrediKarti(limit, guncelBorc: 0.0);
51         krediKartlari.add(yeniKart);
52         System.out.println("Kredi kartı başarıyla eklendi. Limit: " + limit);
53     }
54
55     // Proje isterindeki güvenlik kontrolü: Bakiye sıfırdan büyükse hesabı silmesine izin vermiyorum.
56     public void hesapSil(BankaHesabi hesap) {
57         if (hesap.getBakiye() > 0) {
58             System.out.println(x: "Lütfen öncelikle bakiyenizi başka bir hesaba aktarınız.");
59         } else {
60             hesaplar.remove(hesap);
61             System.out.println(x: "Hesap başarıyla silindi.");
62         }
63     }
64
65     // Güvenlik kontrolü: Borç varken kartın silinmesini engelliyorum.
66     public void krediKartiSil(KrediKarti kart) {
67         if (kart.getGuncelBorc() > 0) {
68             System.out.println(x: "Lütfen öncelikle borç ödemesi yapınız.");
69         } else {
70             krediKartlari.remove(kart);
71             System.out.println(x: "Kredi kartı başarıyla silindi.");
72         }
73     }
74
75     // Getter metodları
76     public String getMusteriNumarasi() { return musterNumarasi; }
77     public ArrayList<BankaHesabi> getHesaplar() { return hesaplar; }
78     public ArrayList<KrediKarti> getKrediKartlari() { return krediKartlari; }
79
80     @Override
81     public String toString() {
82         return super.toString() + ", Müşteri No: " + musterNumarasi;
83     }
84 }

```

Şekil 1.3: Musteri Sınıfı, Hesap/Kart Yönetimi ve Silme Kontrol Algoritmaları

2. com.bank.app.accounts Paketi

Bu paket, banka hesaplarının türlerini ve bu hesaplar üzerinde gerçekleştirilen para yatırma, para çekme ve para transferi gibi temel finansal işlemleri tanımlayan sınıfları içerir.

BankaHesabi.java Sınıfı

Vadesiz ve yatırım hesapları için bir şablon (üst sınıf) görevi gören bu sınıfta, her hesabın sahip olması gereken bakiye ve IBAN bilgileri tanımlanmıştır. Gerçekçi bir banka deneyimi sunmak amacıyla, sınıf nesnesi oluşturulduğunda (constructor içinde) çalışan ve benzersiz bir Türkiye (TR) IBAN'ı üreten rastgele bir algoritma geliştirilmiştir.

```
src > com > bank > accounts > J BankaHesabi.java > BankaHesabi
1  package com.bank.app.accounts;
2
3  import java.util.Random;
4
5  // Tüm hesap türlerinin (Vadesiz ve Yatırım) ortak özelliklerini tutan üst (Base) sınıf.
6  public class BankaHesabi {
7      private String iban;
8      private double bakiye;
9
10     public BankaHesabi(double bakiye) {
11         // IBAN bilgisini dışarıdan almak yerine içeride rastgele üretiyorum.
12         this.iban = rastgeleIbanUret();
13         this.bakiye = bakiye;
14     }
15
16     // Gerçekçi bir TR IBAN simülasyonu yapan metod.
17     private String rastgeleIbanUret() {
18         Random rnd = new Random();
19         return "TR" + (10 + rnd.nextInt(bound: 90)) + "00062000000" + (10000000 + rnd.nextInt(bound: 90000000));
20     }
21
22     public String getIban() { return iban; }
23
24     public double getBakiye() { return bakiye; }
25     public void setBakiye(double bakiye) { this.bakiye = bakiye; }
26
27     @Override
28     public String toString() {
29         return "IBAN: " + iban + ", Bakiye: " + bakiye;
30     }
31 }
```

Şekil 1.4: BankaHesabi Üst Sınıfı ve Otomatik IBAN Üretim Mantığı

2.1 VadesizHesap.java Sınıfı

Müşterilerin günlük finansal akışını yönettiği ana sınıftır ve BankaHesabi sınıfından kalıtım almaktadır. İki farklı hesap arasında para akışını sağlayan paraTransferi metodu ile müşterinin vadesiz bakiyesini kullanarak kredi kartı borcunu ödemesini sağlayan krediKartiBorcOdeme iş mantıkları bakiye kontrolleriyle birlikte bu sınıfta tasarlanmıştır.

```
src > com > bank > accounts > J VadesizHesap.java > VadesizHesap
```

```
1 package com.bank.app.accounts;
2
3 import com.bank.app.cards.KrediKarti;
4
5 // BankaHesabi sınıfından kalıtım alarak vadesiz hesaba özgü işlemleri barındırır.
6 public class VadesizHesap extends BankaHesabi {
7     private String hesapTuru;
8
9     public VadesizHesap(double bakiye) {
10         // super() ile bakiyeyi üst sınıfa (BankaHesabi) iletiyorum.
11         super(bakiye);
12         this.hesapTuru = "Vadesiz";
13     }
14
15     // İki hesap arasında para transferi yapan iş mantığı (Business Logic) metodu.
16     public void paraTransferi(BankaHesabi aliciHesap, BankaHesabi gonderenHesap, double miktar) {
17         // Gönderen hesaptaki paranın, gönderilecek tutara yetip yetmediğini kontrol ediyorum.
18         if (gonderenHesap.getBakiye() >= miktar) {
19             // Yeterliyse gönderenden parayı düşüp, alıcıya ekliyorum.
20             gonderenHesap.setBakiye(gonderenHesap.getBakiye() - miktar);
21             aliciHesap.setBakiye(aliciHesap.getBakiye() + miktar);
22             System.out.println("Başarılı! " + miktar + " TL transfer edildi.");
23         } else {
24             System.out.println(x: "Hata: Yetersiz bakiye!");
25         }
26     }
27
28     // Vadesiz hesaptaki parayla kredi kartı borcunu ödeyen metod.
29     public void krediKartiBorcOdeme(KrediKarti kart, double miktar) {
30         // Hesapta yeterli bakiye var mı kontrolü yapıyorum.
31         if (this.getBakiye() >= miktar) {
32             // Hesaptan parayı düşüyorum.
33             this.setBakiye(this.getBakiye() - miktar);
34
35             // Kartın güncel borcunu ödenen miktar kadar azaltıyorum.
36             kart.setGuncelBorc(kart.getGuncelBorc() - miktar);
37             System.out.println("Başarılı! Kredi kartı borcunuzdan " + miktar + " TL ödendi.");
38         } else {
39             System.out.println(x: "Hata: Yetersiz bakiye!");
40         }
41     }
42
43     public String getHesapTuru() { return hesapTuru; }
44     public void setHesapTuru(String hesapTuru) { this.hesapTuru = hesapTuru; }
45
46     @Override
47     public String toString() {
48         return super.toString() + ", Hesap Türü: " + hesapTuru;
49     }
50 }
```

Şekil 1.5: VadesizHesap Sınıfı ve Hesaplar Arası Para Transferi Metodu

2.2 YatirimHesabi.java Sınıfı

Kullanıcının birikim yapmasını temsil eden, BankaHesabi'ndan miras alan daha sade bir sınıftır. İçerisinde bakiyeyi güvenli bir şekilde artıran paraEkle ve hesapta çekilmek istenen tutar kadar para olup olmadığını kontrol ederek işlem yapan paraCek metotlarını barındırır.

```

src > com > bank > accounts > J YatirimHesabi.java > YatirimHesabi
1  package com.bank.app.accounts;
2
3  // BankaHesabi sınıfından miras alan Yatırım Hesabı modeli.
4  public class YatirimHesabi extends BankaHesabi {
5      private String hesapTuru;
6
7      public YatirimHesabi(double bakiye) {
8          super(bakiye);
9          this.hesapTuru = "Yatırım";
10     }
11
12     // Yatırım hesabına manuel olarak para ekleme işlemi.
13     public void paraEkle(double miktar) {
14         if(miktar > 0) {
15             this.setBakiye(this.getBakiye() + miktar);
16             System.out.println("Yatırım hesabınıza " + miktar + " TL eklendi. Yeni bakiye: " + this.getBakiye());
17         }
18     }
19
20     // Yatırım hesabından para çekme metodu.
21     public void paraCek(double miktar) {
22         // Çekilmek istenen miktar hesaptaki paradan büyük olmamalı.
23         if(this.getBakiye() >= miktar) {
24             this.setBakiye(this.getBakiye() - miktar);
25             System.out.println("Yatırım hesabınızdan " + miktar + " TL çekildi. Yeni bakiye: " + this.getBakiye());
26         } else {
27             System.out.println(x: "Hata: Yatırım hesabınızda yeterli bakiye bulunmuyor!");
28         }
29     }
30
31     public String getHesapTuru() { return hesapTuru; }
32     public void setHesapTuru(String hesapTuru) { this.hesapTuru = hesapTuru; }
33
34     @Override
35     public String toString() {
36         return super.toString() + ", Hesap Türü: " + hesapTuru;
37     }
38 }

```

Şekil 1.6: YatirimHesabi Sınıfı, Para Ekleme ve Bakiye Kontrollü Para Çekme İşlemleri

3. com.bank.app.cards Paketi

Bu paket, bankacılık sisteminin kredi modülünü oluşturur ve müşterilere tanımlanan kredi kartlarının iş mantığını (harcama, borçlanma ve limit takibi) barındırır.

3.1 KrediKarti.java Sınıfı

Müşterinin sahip olduğu kredi kartını modelleyen bu sınıfta; kart numarası, limit, güncel borç ve kullanılabilir limit bilgileri tutulmaktadır. Kart numaralarının 16 haneli olması kuralını sağlamak için Random kütüphanesiyle özel bir simülasyon metodu geliştirdim. Sınıfın en kritik algoritması ise setGuncelBorc metodunun içindedir; bu metot sayesinde karta bir borç yansıtıldığında veya borç ödendiğinde kullanılabilirLimit değeri (Limit - Güncel Borç mantığıyla) anlık ve otomatik olarak güncellenmektedir.


```

src > com > bank > cards > J KrediKarti.java > KrediKarti
1  package com.bank.app.cards;
2
3  import java.util.Random;
4
5  public class KrediKarti {
6      private String kartNumarasi;
7      private double limit;
8      private double guncelBorc;
9      private double kullanilabilirLimit;
10
11     public KrediKarti(double limit, double guncelBorc) {
12         // Proje isterine uygun olarak kart numarasını random oluştuyorum.
13         this.kartNumarasi = rastgeleKartNoUret();
14         this.limit = limit;
15         this.guncelBorc = guncelBorc;
16         // Kullanılabilir limit, toplam limitten güncel borcun çıkarılmasıyla bulunur.
17         this.kullanilabilirLimit = limit - guncelBorc;
18     }
19
20     // 16 haneli rastgele kredi kartı numarası üreten simülasyon metodu.
21     private String rastgeleKartNoUret() {
22         Random rnd = new Random();
23         long randomNo = 1000000000000000L + (long)(rnd.nextDouble() * 8999999999999999L);
24         return String.valueOf(randomNo);
25     }
26
27     public String getKartNumarasi() { return kartNumarasi; }
28
29     public double getLimit() { return limit; }
30     public void setLimit(double limit) { this.limit = limit; }
31
32     public double getGuncelBorc() { return guncelBorc; }
33
34     // Borç değerini her güncellediğimde, kullanılabilir limitin de matematiksel olarak
35     // eş zamanlı güncellenmesini bu Setter metodu içinde garanti altına alıyorum.
36     public void setGuncelBorc(double guncelBorc) {
37         this.guncelBorc = guncelBorc;
38         this.kullanilabilirLimit = this.limit - this.guncelBorc;
39     }
40
41     public double getKullanilabilirLimit() { return kullanilabilirLimit; }
42
43     @Override
44     public String toString() {
45         return "Kart No: " + kartNumarasi + ", Limit: " + limit + ", Güncel Borç: " + guncelBorc + ", Kullanılabilir Limit: " + kullanilabilirLimit;
46     }
47 }

```

Şekil 1.7: KrediKarti Sınıfı Kod Yapısı ve Dinamik Limit Hesaplama Algoritması

4. com.bank.app.service Paketi

Bu paket, sistemdeki veritabanı veya operasyon merkezi gibi çalışır. Nesneler (müşteri ve personel) arasındaki atamaların, listelemelerin ve sistem kayıtlarının yapıldığı servis katmanıdır.

4.1 BankaService.java Sınıfı

Sisteme dahil olan tüm personel ve müşterilerin ana listelerini (ArrayList) barındıran yönetici sınıftır. Bu sınıfın içerisinde yer alan müşteriTemsilcisineAta metodu, sisteme yeni eklenen bir müşteriyi güvenli bir şekilde ilgili banka personeline atamak ve iki nesneyi birbirine bağlamak (ilişkilendirmek) amacıyla tasarlanmıştır.

```

src > com > bank > service > J BankaService.java > BankaService
1 package com.bank.app.service;
2
3 import com.bank.app.people.BankaPersoneli;
4 import com.bank.app.people.Musteri;
5 import java.util.ArrayList;
6
7 // Bu sınıf sistemdeki genel iş mantığını yöneten "Yönetici" veya "Veritabanı" görevi görüyor.
8 public class BankaService {
9
10     // Bankadaki tüm kayıtlı personelleri ve müşterileri tuttuğum listeler.
11     private ArrayList<BankaPersoneli> personeller;
12     private ArrayList<Musteri> musteriler;
13
14     public BankaService() {
15         this.personeller = new ArrayList<>();
16         this.musteriler = new ArrayList<>();
17     }
18
19     // Sisteme genel personel kaydı yapan metot.
20     public void personelEkle(BankaPersoneli personel) {
21         personeller.add(personel);
22         System.out.println("Sisteme yeni personel eklendi: " + personel.getAd() + " " + personel.getSoyad());
23     }
24
25     // Sisteme genel müşteri kaydı yapan metot.
26     public void musteriTemsilcisineAta(Musteri musteriler) {
27         musteriler.add(musteriler);
28         System.out.println("Sisteme yeni müşteri eklendi: " + musteriler.getAd() + " " + musteriler.getSoyad());
29     }
30
31     // Sisteme eklenen müşteriyi, belirli bir banka personeline (temsilci) atama metodu.
32     public void musteriTemsilcisineAta(BankaPersoneli personel, Musteri musteriler) {
33         // Olası hataları engellemek için nesnelerin null olup olmadığını kontrol ediyorum.
34         if (personel != null && musteriler != null) {
35             personel.musteriTemsilcisineAta(musteriler);
36             System.out.println("Atama Başarılı: " + musteriler.getAd() + " isimli müşteri, " + personel.getAd() + " temsilcisine atandı.");
37         }
38     }
39 }

```

Şekil 1.8: BankaService Sınıfı ve Personel-Müşteri Eşleştirme İş Mantığı (Business Logic)

5. com.bank.app.main Paketi

Uygulamanın giriş noktasıdır ve yazılan tüm modüler yapıların (sınıflar, metotlar ve güvenlik algoritmalarının) tek bir senaryo altında birleşip test edildiği pakettir.

5.1 Main.java Sınıfı

Projenin çalıştırıldığı ve ödev yönergesinde istenen 7 adımlık test senaryosunun kurgulandığı ana sınıftır. Uygulamayı sadece statik kodların çalıştığı bir yapıdan çıkarıp, gerçek bir terminal yazılımına dönüştürmek amacıyla Java'nın Scanner kütüphanesini kullandım. Bu sayede program çalıştığında; personel adı, müşteri bilgileri, yatırılacak tutar ve transfer edilecek miktar gibi veriler doğrudan kullanıcıdan (klavyeden) dinamik olarak alınmaktadır.

```

src > com > bank > main > J Main.java > Main
1 package com.bank.app.main;
2
3 import com.bank.app.accounts.BankaHesabi;
4 import com.bank.app.accounts.VadesizHesap;
5 import com.bank.app.accounts.YatirimHesabi;
6 import com.bank.app.cards.KrediKarti;
7 import com.bank.app.people.BankaPersoneli;
8 import com.bank.app.people.Musteri;
9 import com.bank.app.service.BankaService;
10 import java.util.Scanner;
11
12 // Projenin çalıştığı, Scanner ile dinamik verilerin alındığı test sınıfı.
13 public class Main {
14     Run | Debug
15     public static void main(String[] args) {
16         // Kullanıcıdan veri almak için Scanner nesnesini başlatıyorum.
17         Scanner scanner = new Scanner(System.in);
18         BankaService bankaServisi = new BankaService();
19
20         System.out.println(x: "=== BTÜ BANKA OTOMASYON SİSTEMİNE HOŞ GELDİNİZ ===");
21
22         // ---- ADIM 1: Personel ve Müşteri Kaydı ----
23         System.out.print(s: "Sisteme eklenecek personelin adı: ");
24         String pAd = scanner.nextLine();
25         System.out.print(s: "Personelin soyadı: ");
26         String pSoyad = scanner.nextLine();
27         // Konsoldan aldığım isimle dinamik mail oluşturup yeni personel nesnesi yaratıyorum.
28         BankaPersoneli personel = new BankaPersoneli(pAd, pSoyad, pAd.toLowerCase() + "@banka.com", telefonNumarasi: 5550001);
29         bankaServisi.personelEkle(personel);
30
31         System.out.print(s: "\nKaydı yapılacak müşterinin adı: ");
32         String mAd = scanner.nextLine();
33         System.out.print(s: "Müşterinin soyadı: ");
34         String mSoyad = scanner.nextLine();
35         Musteri musterisi = new Musteri(mAd, mSoyad, mAd.toLowerCase() + "@mail.com", telefonNumarasi: 5550002);
36         bankaServisi.musteriEkle(musterisi);
37
38         // Müşteriyi personele atama işlemi
39         bankaServisi.musteriTemsilcisineAta(personel, musterisi);
40
41         // ---- ADIM 2: Hesap Açma ve Para Yatırma ----
42         System.out.println(x: "\n--- Hesap Açma İşlemleri ---");
43         musterisi.hesapEkle(hesapTuru: "Vadesiz");
44         musterisi.hesapEkle(hesapTuru: "Yatirim");
45
46         // Polimorfizm kullanarak nesneleri ilgili hesap tiplerine dönüştürüyorum (Casting).
47         VadesizHesap vadesiz = (VadesizHesap) musterisi.getHesaplar().get(index: 0);
48         YatirimHesabi yatirim = (YatirimHesabi) musterisi.getHesaplar().get(index: 1);
49
50         System.out.print(s: "Vadesiz hesaba başlangıç için yatırılacak tutar: ");
51         double vBakiye = scanner.nextDouble();
52         vadesiz.setBakiye(vBakiye);
53
54         System.out.print(s: "Yatırım hesabına başlangıç için yatırılacak tutar: ");
55         double yBakiye = scanner.nextDouble();
56         yatirim.paraEkle(yBakiye);
57
58         // ---- ADIM 3: Para Transferi ----
59         System.out.println(x: "\n--- Para Transferi ---");
60         System.out.print(s: "Vadesiz hesaptan Yatırım hesabına ne kadar para gönderilsin? ");
61         double transferTutari = scanner.nextDouble();
62         // Transfer iş mantığını tetikliyorum.
63         vadesiz.paraTransferi(yatirim, vadesiz, transferTutari);
64
65         // ---- ADIM 4: Kredi Kartı İşlemleri ----
66         System.out.println(x: "\n--- Kredi Kartı Tanımlama ---");
67         System.out.print(s: "Kredi kartı limitini belirleyin: ");
68         double limit = scanner.nextDouble();
69         musterisi.krediKartiEkle(limit);
70         KrediKarti kart = musterisi.getKrediKartlari().get(index: 0);
71
72         // Test için sanal bir alışveriş borcu yansıtıyorum.
73         System.out.print(s: "Karta yansıtılacak (harcanan) borç tutarı: ");
74         double borc = scanner.nextDouble();
75         kart.setGuncelBorc(borc);
76
77         System.out.print(s: "Vadesiz hesap bakiyenizden, kredi kartı borcunuzun ne kadarını ödemek istersiniz? ");
78         double odeme = scanner.nextDouble();
79         vadesiz.krediKartiBorcOdeme(kart, odeme);
80
81         // ---- ADIM 5: Silme Kontrolleri (Güvenlik Algoritmaları) ----
82         System.out.println(x: "\n--- Sistemden Silme Testleri ---");
83
84         System.out.println(x: "-> Borcu olan kart silinmeye çalışılıyor...");
85         musterisi.krediKartiSil(kart); // Borç sıfırlanmadığı için hata uyarısı vermeli.
86
87         System.out.println(x: "\n-> İçinde bakiyesi olan yatırım hesabı silinmeye çalışılıyor...");
88         musterisi.hesapSil(yatirim); // Hesapta para olduğu için hata uyarısı vermeli.
89
90         System.out.println(x: "\nİşlemler tamamlandı. Program sonlandırılıyor.");
91         // Scanner nesnesini kapatarak bellek (memory) sızıntısını önliyorum.
92         scanner.close();
93     }

```

Şekil 1.9: Main Sınıfı İçerisinde Scanner Kullanımı ve Dinamik Veri Alma Yapısı

ÖRNEK KULLANIM SENARYOSU VE UYGULAMA ÇIKTILARI

Bu bölümde, Main.java üzerinden başlatılan ve kullanıcıdan Scanner ile verilerin alındığı terminal ekran çıktıları, proje isterlerinde belirtilen sıraya göre sunulmuştur.

Sisteme Kayıt ve Hesap Açılışı

Kullanıcıdan alınan isim ve soyisim bilgileriyle sistemde personel ve müşteri oluşturulmuş, müşteri ilgili temsilciye atanmıştır. Ardından müşteriye ait vadesiz ve yatırım hesapları açılarak klavyeden girilen tutarlar bu hesaplara bakiye olarak tanımlanmıştır

```
bilgisayar@bilgisayars-MacBook-Air src % /usr/bin/env /Library/Java/JavaVirtualMachines/jdk-25.jdk/Contents/Home/bin/java -XX:+ShowCodeDetailsInExceptionMess
ilgisayar/Library/Application\ Support/Code/User/workspaceStorage/db5eeaa6a2725774697324ee4be91c56/redhat.java/jdt_ws/jdt.ls-java-project/bin com.bank.app.mai
=== BTÜ BANKA OTOMASYON SİSTEMİNE HOŞ GELDİNİZ ===
Sisteme eklenecek personelin adı: Emin Eren
Personelin soyadı: KADIOĞLU
Sisteme yeni personel eklendi: Emin Eren KADIOĞLU

Kaydı yapılacak müşterinin adı: Nyp
Müşterinin soyadı: Proje
Sisteme yeni müşteri eklendi: Nyp Proje
Atama Başarılı: Nyp isimli müşteri, Emin Eren temsilcisine atandı.

--- Hesap Açma İşlemleri ---
Vadesiz hesap başarıyla eklendi.
Yatırım hesabı başarıyla eklendi.
Vadesiz hesaba yatırılacak tutar: 2500
Yatırım hesabına yatırılacak tutar: 3500
Yatırım hesabınıza 3500.0 TL eklendi. Yeni bakiye: 3500.0
```

Şekil 2.0: Kullanıcı Girdileriyle Müşteri Oluşturma ve Hesaplara Başlangıç Bakiyesi Tanımlama Ekranı

Para Transferi ve Kredi Kartı İşlemleri

Kullanıcının belirlediği transfer tutarı (örn: 2500 TL) vadesiz hesaptan yatırım hesabına başarıyla geçirilmiştir. Daha sonra müşteriye kredi kartı tanımlanmış ve yine kullanıcının belirlediği tutar kadar borç vadesiz hesap bakiyesinden ödenmiştir.

```
--- Para Transferi ---
Vadesiz hesaptan Yatırım hesabına ne kadar gönderilsin? 1400
Başarılı! 1400.0 TL transfer edildi.

--- Kredi Kartı Tanımlama ---
Kredi kartı limitini belirleyin: 5000
Kredi kartı başarıyla eklendi. Limit: 5000.0
Karta yansıtılacak borç tutarı: 2400
Kredi kartı borcunun ne kadarını ödemek istersiniz? 1300
Hata: Yetersiz bakiye!
```

Şekil 2.1: Hesaplar Arası Para Transferi ve Vadesiz Bakiyeden Kart Borcu Ödeme İşlemi Çıktısı

Şartlı Silme (Güvenlik) Kontrolleri

Projenin en önemli isterlerinden olan "Silme Kontrolü" algoritmaları test edilmiştir. Sistem, borcu tamamen sıfırlanmamış kredi kartını ve içerisinde bakiye bulunan yatırım hesabını silmeyi reddederek doğru uyarıları ("Lütfen bakiyenizi aktarınız", "Borç ödemesi yapınız") vermiştir.

```
--- Sistemden Silme Testleri ---  
Borcu olan kart siliniyor...  
Lütfen öncelikle borç ödemesi yapınız.  
  
Bakiyesi olan yatırım hesabı siliniyor...  
Lütfen öncelikle bakiyenizi başka bir hesaba aktarınız.  
  
İşlemler tamamlandı. Program sonlandırılıyor.
```

Şekil 2.2: Sistem Güvenliği: Bakiyesi Olan Hesabın ve Borcu Olan Kartın Silinmesinin Engellenmesi

KARŞILAŞILAN ZORLUKLAR VE ÇÖZÜM SÜREÇLERİ

Projenin geliştirme aşamasında özellikle Java'nın katı paket yapısı ve izin hiyerarşisi (com.bank.app...) konusunda bazı teknik zorluklarla karşılaşılmıştır. Klasörlerin işletim sistemi düzeyindeki hiyerarşisi ile Java paket tanımlamaları arasındaki uyumsuzluktan kaynaklanan "Package does not exist" hataları, izin yapısı projenin kök dizinine (src) göre, iç içe geçmiş klasörler halinde yeniden düzenlenerek çözüme kavuşturulmuştur.

Buna ek olarak, Main sınıfında Scanner kütüphanesini kullanırken (String girdilerinden hemen sonra sayısal girdiler alırken) yaşanan satır atlama (buffer) problemleri, değişken sıralamaları optimize edilerek giderilmiştir. Tüm bu hata ayıklama süreçleri, terminal ortamına ve Java'nın dosya mimarisine olan hakimiyetin pekişmesini sağlamıştır.

PROJE BAĞLANTILARI

Geliştirilen uygulamanın tüm kaynak kodlarına ve çalıştırılma senaryosunun detaylı olarak açıklandığı sunum videosuna aşağıdaki bağlantılar üzerinden ulaşılabilir:

- **GitHub Kaynak Kod Deposu:** https://github.com/emin-eren-kadioglu/NYP_Bank_Service_Project
- **Proje Sunumu ve Test Videosu (YouTube):** <https://youtu.be/IIHicCQppL8>

Sonuç: Bu proje çalışması sayesinde; Nesneye Yönelik Programlama (OOP) dersi kapsamında öğrenilen kalıtım, kapsülleme ve polimorfizm gibi teorik kavramların, birbirine entegre sınıflar (paketler) aracılığıyla gerçek bir senaryoda nasıl başarılı bir şekilde çalıştığı pratik edilerek öğrenilmiştir.